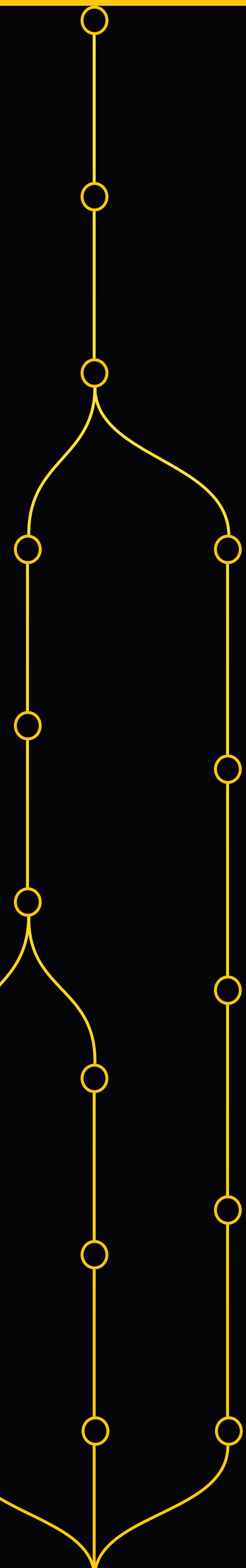
Mission Control

Track C#

August 31, 2026



Innovation Campus

Харківський політехнічний інститут
1885
НТУ

Contents

Engage 2

Investigate 3

Act 5

Share 9

Engage

DESCRIPTION

Congratulations, Challenger.

You've built a powerful and flexible travel system – now it's time to make it usable. In this challenge, you'll wrap your logic in a graphical interface, creating a full-featured desktop application that allows users to plan and manage trips without writing a single line of code.

The goal is not just to build something functional, but to make it intuitive, clean, and user-centered. How will users interact with the system? How should errors be handled visually? What feedback will they receive from the interface?

Users will be able to create travelers, view and edit routes, and calculate the shortest path between cities – all visually. You'll also handle file loading, saving, and error feedback through friendly dialogs.

This is where your system becomes a product – and where users take control.

BIG IDEA

Connecting code and users through visual design and interactivity.

ESSENTIAL QUESTION

How can we design an interface that transforms a system into an experience?

CHALLENGE

Create a desktop application using C# and a GUI framework (Windows Forms or WPF) that allows users to interact with your travel system. The app should support creating and managing travelers, editing routes, saving/loading data, and handling errors in a user-friendly way.

Investigate

GUIDING QUESTIONS

As you move into building graphical user interfaces, take time to explore the following questions. They will help you understand how to design, structure, and connect your application to real user interaction:

- What is a GUI (Graphical User Interface)?
- What are the differences between Windows Forms and WPF in C#?
- What is the event-driven programming model?
- How do button clicks and user actions trigger code in a form?
- How can you connect UI elements to your classes and data?
- What are the most important principles of user interface (UI) and user experience (UX) design?
- How do you handle exceptions and show error messages to users?
- What are common controls in Windows Forms (e.g., TextBox, ListBox, ComboBox, Button)?
- How can you open/save files using dialogs?
- What are `DataBinding`, `Form`, and `UserControl`?
- How should the logic layer be separated from the UI?

GUIDING ACTIVITIES

Prepare yourself by completing the following activities. You can do this individually or in small groups. Experiment freely – GUI development is visual, interactive, and best learned by doing:

- Learn how to create a starter Windows Forms / WPF project in Visual Studio.
- Create a simple Windows Forms or WPF app that shows a message when you click a button. You can use this tutorial:

  Create a Windows Forms app in Visual Studio with C#

- Learn how to place and configure UI elements (buttons, labels, inputs, listboxes)
- Practice reading and writing values from input fields (`TextBox`) and showing results in labels or lists
- Explore how to use `OpenFileDialog` and `SaveFileDialog` to work with files
- Connect a simple UI to a class (e.g., create a form to input a traveler's name and display it)
- Try handling exceptions and showing them in a message box
- Plan the structure of your application:
  - What screens (forms) are needed?
  - What information will each screen display?
  - What actions will each button perform?
  - Sketch a basic wireframe layout on paper or digitally (Canva, Figma, etc.)
- Think like a user: what would make this app easy to use?

ANALYSIS

Analyze your findings. What conclusions have you made after completing guiding questions and activities? In addition to your thoughts and conclusions, here are some more analysis results.

- Challenge has to be carried out by an entire team
- Discuss with your team the plan for implementing the chosen idea.
- Despite your ambitious plans, keep in mind that your project must be compiled in less than 30 seconds. Otherwise, the challenge is considered failed.
- Research the tools and technologies that will help you realize your solution.
- The challenge must be performed in C#.
- Discuss your app design. Create logic for how the user should use your product. In other words, make a brainstorm with your team and write down a detailed script of how the user will use your app.
- Your project can be built with one build command `dotnet build SmartTravelPlanner/Project.sln -c Release -m`
- Do not forget that the user interface should be easy to use.
- Submit your files using the layout described in the story. Only useful files allowed, garbage shall not pass!
- Proper resource management is required: no memory leaks, no undisposed handles, and no lingering event subscriptions. Otherwise, the challenge is considered incomplete.
- Complete the challenge according to the rules specified in the Microsoft's C# identifier naming rules and conventions та Common C# code conventions.
- The solution will be checked and graded by students like you. Peer-to-Peer learning.
- If you have any questions or don't understand something, ask other students, Google it or ask AI chatbot like ChatGPT, Claude, Gemini. However, remember to verify the information you receive.
- AI tools (e.g., ChatGPT, Claude, Gemini, or similar) may be used as learning assistants to ask questions, explore concepts, understand documentation, debug errors, and receive guidance. They should support your learning process but not replace it. Generating complete assignment solutions or ready-to-submit code with AI is prohibited. Your final submission must be written and understood by you. Remember that you have ownership of, accountability for, and responsibility for the code you commit.

Act

NAME

Base Class Refactor

DIRECTORY

`t00/`

SUBMIT

`SmartTravelPlanner/{Project.sln, other project directories and files}, diagrams/{usecase.[png,jpg], activity.[png,jpg], sequence.[png,jpg]}, README.md`

BINARY

`SmartTravelPlanner.exe or SmartTravelPlanner depending on the system on which the build is performed`

DESCRIPTION

Build a graphical application for planning smart travel routes. The app should allow the user to:

- Create and manage a traveler (name, current location)
- Load a map of cities and distances from a `.txt` file
- Enter a destination city
- Automatically find and display the shortest path between the traveler's location and the destination
- Manually add or remove a pair of cities with distance from the map, if needed.
- Display the traveler's full route and total travel distance
- Save/load the traveler’s state to/from a `.json` file
- Show user-friendly error messages when something goes wrong (e.g., file not found, invalid input)

You may extend the application’s functionality, but the required part must be implemented first.

TECHNICAL REQUIREMENTS

Your application must:

- Use Windows Forms or WPF or Avalonia UI
- Contain the following UI components:
    - Input fields for traveler name and location
    - A “Load Map” button to load a city graph from a `.txt` file
    - A destination input field and “Plan Route” button
    - Display for planned route and total distance
    - A list or textbox showing all route cities
    - Buttons: `Save`, `Load`, `Clear Route`, `Exit`
- Implement the graph logic using your `CityGraph` class from Challenge 2
- Use Dijkstra’s algorithm (or similar) to calculate the shortest route
- Support serialization/deserialization of `Traveler` using JSON
- Validate user input and handle file errors

ERROR HANDLING

Display a message box for the following scenarios:

- Map file not found or invalid format
- Destination not reachable or not in map
- Empty traveler or destination field
- Destination not reachable
- Invalid `.json` file during loading

EXAMPLE WORKFLOW

1. User opens app
2. Enters: `Alice`, `Kyiv` and clicks `Create Traveler`
3. Loads map file `map.txt` (containing city connections)
4. Enters destination: Berlin → clicks Plan Route
5. App shows route: `Kyiv → Lviv → Warsaw → Berlin`
6. Displays total distance: `1240 km`
7. Clicks `Save` → `traveler.json` created
8. Next session: clicks `Load` → traveler loaded

UML Diagrams

As part of this challenge, you must document the system using three UML diagrams. These diagrams will help visualize how your application works from different perspectives: user interaction, process logic, and system communication.

Each diagram must be submitted as an image file into diagrams directory. These diagrams should be added to the README.md file as links with corresponding heading names.

Diagram quality requirements:

- Minimum resolution: 1280×720 px (HD)
- Diagrams must be readable without zooming
- Avoid pixelation or blurry exports
- Preferred formats: .png or .jpg with clear text and lines

USE CASE Diagram

File name: `usecase.png`

Purpose:
Illustrates the interactions between the user and the system — what actions the user can perform and how they relate to core functionality.

Requirements:

- Show at least one actor: User
- Use cases: `Create Traveler`, `Load Map`, `Plan Route`, `Save Traveler`, `Load Traveler`, `View Route`, `Exit`
- Use standard UML symbols (ovals for use cases, lines connecting actors)

ACTIVITY Diagram

File name: `activity.png`

Purpose:
Describes the internal workflow of the application when the user plans a route.

Requirements:

- Start from action like `User enters data`
- Show steps such as: `Validate input` → `Load map` → `Find path` → `Update route`
- Include decisions (diamonds) and possible error flows (e.g., “File not found”, “Invalid destination”)
- End with successful path planned or error handled

SEQUENCE Diagram

File name: `sequence.png`

Purpose:
Demonstrates how objects in your system communicate over time during the route planning process.

Requirements:

- Include objects: `MainForm`, `Traveler`, `CityGraph`
- Messages like: `PlanRoute(destination)` → `FindShortestPath()` → `UpdateRoute()`
- Show the timeline and order of interactions
- Include return values or results where relevant

Submission Notes

Place all three diagrams in your project root directory.
Ensure the file names are exactly as specified:

- `usecase.png`
- `activity.png`
- `sequence.png`

You may create the diagrams using any tool (draw.io, Lucidchart, StarUML, etc.) but they must be clearly labeled and readable.

Share

DESCRIPTION

Last but not least, the final stage of your work is to publish it on LinkedIn in a form of post. This step isn't just about showcasing your work — it's a crucial part of Challenge Based Learning framework. During this stage, you will discover ways of getting external evaluation and feedback on your work. Analyzing your process helps you to understand your strengths and areas for improvement, while sharing your insights invites valuable feedback from peers and professionals.

Pay attention to details:

- aim to make your post both informative and reflective
- start by briefly introducing the challenge and its purpose
- highlight the key challenges you faced and the technologies or solutions you used to overcome them
- share any significant findings or lessons learned during the process
- conclude with a reflection on how this experience has prepared you for future work and invite others to share their thoughts

Helpful tools:

- Canva - a good way to visualize your data
- QuickTime or OBS - an easy way to capture your screen, record video or audio

Don’t forget to tag your post with #InnovationCampusKhPI to connect with the community! Follow and mention via @ Innovation Campus of NTU "KhPI"!